

Pruebas de software

Informe de analisis SonarQube

Nombre del proyecto: EcoSense

Integrantes: Billy Martinez - Bastian Lagos

Fecha de ejecucion: 27/05/2026

Configuracion y ejecucion

Analisis ejecutado con SonarQube Community Edition en Docker y SonarScanner CLI sobre app/src/main/kotlin.

Elemento	Resultado
SonarQube	9.9.8.100196
SonarScanner CLI	8.0.1.6346
Project key	ecosense
Archivos indexados	55
Kotlin source files analizados	44
NCLOC	5373
Quality Gate	OK

Resultados obtenidos

Metrica	Resultado obtenido	Interpretacion del equipo
Bugs	0	SonarQube no detecto defectos clasificados como bugs. Esto favorece la confiabilidad, aunque no reemplaza pruebas funcionales ni revision manual.
Vulnerabilities	0	No se detectaron vulnerabilidades en el codigo analizado. El resultado es positivo para seguridad, pero depende de las reglas activas y del alcance analizado.
Code Smells	20	Existen problemas de mantenibilidad. La mayoría corresponde a complejidad cognitiva, parametros excesivos e imports sin uso; no bloquean ejecucion, pero elevan deuda tecnica.
Coverage	0.0%	SonarQube no recibio reporte JaCoCo/Kover. Aunque el proyecto tiene pruebas, la cobertura no esta integrada al analisis, por lo que el riesgo real no queda medido.
Duplications	0.0%	No se detectaron bloques duplicados relevantes. Esto indica buena reutilizacion general y baja repeticion estructural en el codigo fuente analizado.
Maintainability Rating	A (1.0)	La deuda tecnica total mantiene una calificacion A. Aun asi, los 20 code smells deben revisarse antes de que la complejidad aumente.
Security Rating	A (1.0)	La calificacion de seguridad es la mejor disponible porque no hay vulnerabilidades abiertas ni hotspots de seguridad detectados.
Reliability Rating	A (1.0)	La confiabilidad queda en A porque no se detectaron bugs. El resultado debe complementarse con cobertura medible y pruebas de regresion.

Analisis de hallazgos criticos

Hallazgo critico	Tipo	Severidad	Archivo o modulo afectado	Explicacion del problema
Cobertura reportada en 0.0%	Coverage	Alta	Proyecto completo / JaCoCo XML Report Importer	SonarQube detecto que no se importo ningun reporte de cobertura. El log indica: No report imported, no coverage information will be imported. Esto impide evaluar que porcentaje del codigo esta protegido por pruebas automatizadas.
Complejidad cognitiva excesiva en formulario de reciclaje	Code Smell kotlin:S3776	CRITICAL	app/src/main/kotlin/com/ecosense/screen/RecycleFormScreen.kt:73	SonarQube reporta complejidad 36 sobre un maximo permitido de 15. La pantalla concentra parsing de QR, estado UI, permisos, camara y envio del formulario, lo que dificulta mantenimiento y aumenta riesgo de errores.
Funcion de navegacion con demasiados parametros	Code Smell kotlin:S107	MAJOR	app/src/main/kotlin/com/ecosense/AppNavHost.kt:74	AppNavigation recibe 9 parametros cuando la regla permite 7. Esto hace mas fragil el contrato entre componentes Compose y complica cambios futuros de configuracion visual o navegacion.
Literal duplicado en servicio de grupos	Code Smell kotlin:S1192	CRITICAL	app/src/main/kotlin/com/ecosense/service/GrupoApplicationService.kt	El primer analisis detecto el literal "Usuario no encontrado" repetido tres veces. Esta repeticion puede provocar mensajes inconsistentes si se modifica solo una ocurrencia.

Acciones tomadas o propuestas

Hallazgo	Accion tomada o propuesta	Estado	Evidencia
Cobertura 0.0%	Configurar generacion de reporte XML con JaCoCo o Kover para tests unitarios JVM y declarar sonar.coverage.jacoco.xmlReportPaths en sonar-project.properties.	Pendiente	docs/sonarqube-metrics.json muestra coverage = 0.0; log del scanner indica que no se importo reporte de cobertura.
Complejidad en RecycleFormScreen	Propuesta: extraer parsing QR, manejo de permisos/camara y side effects de Toast/navegacion a funciones o ViewModel; dividir la pantalla en composables mas pequenos.	Pendiente	docs/sonarqube-issues.json mantiene issue abierto kotlin:S3776 en RecycleFormScreen.kt:73.
AppNavigation con 9 parametros	Propuesta: crear un data class de configuracion visual o un objeto de acciones para reducir el numero de parametros expuestos por la funcion.	Pendiente	docs/sonarqube-issues.json mantiene issue abierto kotlin:S107 en AppNavHost.kt:74.
Literal duplicado Usuario no encontrado	Se aplico constante USER_NOT_FOUND en GrupoApplicationService y se reemplazaron las tres ocurrencias duplicadas.	Resuelto	docs/sonarqube-resolved-issues.json marca kotlin:S1192 como CLOSED/FIXED. Code smells bajaron de 22 a 20 tras la nueva ejecucion.
Bloque vacio en RecycleFormScreen	Se reemplazo el else vacio por estados explicitos Idle y Uploading con Unit.	Resuelto	docs/sonarqube-resolved-issues.json marca kotlin:S108 como CLOSED/FIXED.

Evidencia reproducible

```
powershell -ExecutionPolicy Bypass -File .\scripts\run-sonarqube-analysis.ps1
docker run --rm -v ${PWD}:/usr/src sonarsource/sonar-scanner-cli:latest
-Dsonar.host.url=http://172.17.0.1:9000 -Dsonar.login=admin -Dsonar.password=admin
.\gradlew.bat :app:testDebugUnitTest
```

Archivos de evidencia: docs/sonarqube-metrics.json, docs/sonarqube-issues.json, docs/sonarqube-resolved-issues.json y docs/sonarqube-qualitygate.json.

Conclusion

El proyecto no presenta bugs ni vulnerabilidades detectadas y tiene 0.0% de duplicacion. El principal riesgo esta en cobertura no importada y mantenibilidad por complejidad en pantallas Compose. Se corrigieron dos code smells y se dejaron acciones propuestas para los hallazgos restantes.